Boyer-Moore Voting Algorithm

Majority Element ko O(n) time aur O(1) extra space mein find karna

Topic: LeetCode 169 — Majority Element
Language: Java (IntelliJ IDEA format)
Explanation: Simple Roman Hindi + step-by-step dry run

1. Problem ko samjho

Humein ek integer array nums diya hota hai. Humein woh element return karna hai jo array mein total elements ke half se zyada baar aata hai. Isi element ko majority element kehte hain.

Majority condition: frequency > floor(n / 2)

Example: nums = [2, 2, 1, 1, 1, 2, 2]. Total elements n = 7. floor(7/2) = 3. Element 2 total 4 baar aata hai, jabki 1 total 3 baar. Isliye answer 2 hai.

2. Algorithm ka core idea: Voting + Cancellation

Socho har element ek vote de raha hai. Hum ek possible candidate maintain karte hain. Jab current element candidate ke equal ho, candidate ka unmatched vote badhta hai. Jab different ho, dono taraf ka ek-ek vote cancel maan lete hain. Agar count zero ho jaaye, toh agla element naya candidate ban sakta hai.

Important: count actual frequency nahi hai. Yeh cancellation ke baad current candidate ke bache hue unmatched votes ka count hai.

3. Do variables ka role

Variable	Meaning
candidate	Filhaal ka possible majority element. Yeh final answer hona zaroori nahi, jab tak problem majority exist hone ki guarantee na de.
count	Current candidate ke cancellation ke baad bache unmatched votes.

4. Algorithm ke steps

1) candidate aur count initialize karo: candidate = 0, count = 0.
2) Array ko left se right traverse karo.
3) Agar count == 0 hai, current num ko candidate banao.
4) Agar num == candidate hai, count++ karo.
5) Warna count-- karo.
6) Loop ke baad candidate return karo (jab majority exist karne ki guarantee ho).

Pseudocode

```
candidate = 0
count = 0

for each num in nums:
```

```
    if count == 0:
        candidate = num

    if num == candidate:
        count++
    else:
        count--

return candidate
```

5. Complete Java Code (IntelliJ IDEA)

```
package Arrays;

public class MajorityElement {

    public static int majorityElement(int[] nums) {

        int candidate = 0;
        int count = 0;

        for (int num : nums) {

            if (count == 0) {
                candidate = num;
            }

            if (num == candidate) {
                count++;
            } else {
                count--;
            }
        }

        return candidate;
    }

    public static void main(String[] args) {

        int[] nums = {2, 2, 1, 1, 1, 2, 2};

        System.out.println(majorityElement(nums));
    }
}
```

Output: 2

6. Complete dry run

Input: [2, 2, 1, 1, 1, 2, 2]. Starting state: candidate = 0, count = 0.

Step	num	candidate	count	Reason
1	2	2	1	count zero tha; 2 candidate bana, phir same hone se +1
2	2	2	2	Same candidate; count +1
3	1	2	1	Different; ek vote cancel, count -1
4	1	2	0	Different; ek aur vote cancel
5	1	1	1	Count zero tha; 1 naya candidate bana, phir +1
6	2	1	0	Different; vote cancel
7	2	2	1	Count zero tha; 2 naya candidate bana, phir +1

Final candidate = 2, isliye answer 2 return hota hai.

7. Majority element survive kyun karta hai?

Maan lo array ka size n hai. Majority element ki frequency n/2 se zyada hoti hai. Baaki sabhi elements ki combined frequency n/2 se kam hoti hai. Cancellation mein ek majority occurrence ko ek non-majority occurrence ke saath pair karke cancel kar sakte hain. Lekin majority occurrences zyada hain, isliye sab non-majority elements cancel hone ke baad bhi majority element ke kuch votes bachenge.

Simple example: [2, 2, 2, 1, 1]. Do 1s, do 2s ko cancel kar sakte hain. Ek 2 bach jaata hai. Isliye 2 majority candidate survive karta hai.

8. Important limitation: candidate ko verify kab karna hai?

Boyer-Moore ek candidate nikalta hai. Agar problem guarantee karti hai ki majority element exist karta hai, toh candidate hi answer hai. Agar guarantee nahi hai, toh ek second traversal karke candidate ki actual frequency count karo. Agar frequency floor(n/2) se zyada nahi hai, toh majority element exist nahi karta.

```
int frequency = 0;

for (int num : nums) {
    if (num == candidate) {
        frequency++;
    }
}

if (frequency > nums.length / 2) {
    return candidate;
}

// Majority element does not exist
```

9. Complexity analysis

Type	Complexity	Reason
Time	O(n)	Array ko ek baar traverse karte hain; har element par constant work hota hai.
Extra Space	O(1)	Sirf candidate aur count jaise fixed variables use hote hain.

10. Boyer-Moore kis type ki problems mein use hota hai?

LeetCode 169 — Majority Element: frequency > n/2. Standard Boyer-Moore.
LeetCode 229 — Majority Element II: frequency > n/3. Extended Boyer-Moore; maximum 2 candidates maintain hote hain.
Frequency > n/k: Generalized majority-candidate approach, commonly Misra-Gries algorithm se solve hota hai.

11. Common mistakes

• `count` ko actual frequency mat samajhna; yeh unmatched votes hain.
• `if (count == 0)` ke baad same loop iteration mein comparison bhi hota hai; isliye naye candidate ka count 1 ho sakta hai.
• Majority exist hone ki guarantee na ho, toh final candidate verify karna zaroori hai.
• Condition strict hai: more than n/2, equal to n/2 nahi.

12. Quick revision

Same element: count++
Different element: count--
count == 0: current element becomes new candidate
Guaranteed majority: return candidate

No guarantee: verify candidate frequency

Core takeaway: Boyer-Moore Voting Algorithm ka main idea Candidate Selection + Cancellation hai. Yeh majority candidate ko O(n) time aur O(1) extra space mein find karta hai.

Prepared as a study guide for LeetCode 169 — Majority Element.